

浙江大学实验报告

专业：微电子科学与工程
姓名：赵楼晗
学号：3240104430
日期：2026 年 5 月 11 日
地点：紫金港东 4-216

课程名称：电子电路设计实验 I 指导老师：施红军、叶险峰、邓靖靖 成绩：
实验名称：Arduino 实时音频处理实验 实验类型：设计型实验 同组学生姓名：徐莘宁

一、实验目的

- 1、学习掌握用 Arduino UNO 进行实时音频信号处理；
- 2、学习掌握 PCB 电路板的设计和制作；
- 3、学习掌握 Arduino UNO 音频处理扩展板的设计与制作；
- 4、熟悉电流信号转换成电压信号的原理；
- 5、掌握标准电流信号转换成电压信号的设计方法；
- 6、巩固电路原理图的绘制方法与规范，提高电路分析与设计能力；
- 7、巩固 PCB 布局布线的基本原则与设计流程，掌握元器件封装选择、设计规则检查等关键环节；
- 8、通过完整的原理图与 PCB 设计实践，培养工程化设计思维与硬件调试能力。

二、实验任务

2.1 电流电压转换实验

- 熟悉电流-电压转换电路的工作原理，并设计出一个电路，将标准电流信号 $4\text{mA} \sim 20\text{mA}$ 转换为标准电压信号 $0\text{V} \sim 10\text{V}$ ，误差控制在 5% 以内。
- 掌握原理图和 PCB 设计的基本流程，巩固焊接与电路调试。

2.2 Arduino 实时音频处理实验

- 用 Arduino UNO 实现音频信号处理设计，在（1）音频正弦波信号源（2）混响发生器中选择一个电路，并分别加上（3）音频输出放大器，形成一个完整的电路，完成相应器件的选择，制作 Arduino UNO 扩展板。
- 编制与调试相关程序。
- 将制作的扩展板与 Arduino UNO 板组装后，进行系统联调。

三、实验原理

3.1 电流电压转换实验

在自动控制技术中，传感器输出的标准电流信号通常为 $4\text{mA} \sim 20\text{mA}$ ，为了便于进一步处理，需要将其转换为 $0\text{V} \sim 10\text{V}$ 的电压信号。这可以通过工作在线性区的运算放大器来实现。

如图1是一种常见的电流电压转换电路设计思路，其主要由两个运放组成的两级放大电路实现：

第一级放大电路：电阻 R_1 跨接在电流源两端，运放 A_1 采用差分输入，将 $4\text{mA} \sim 20\text{mA}$ 的电流转换为电压。此级放大电路的放大倍数为 1，输出电压 $V_{o1} = -I_i \times R_1$ ，实现了电流到电压的初步转换。

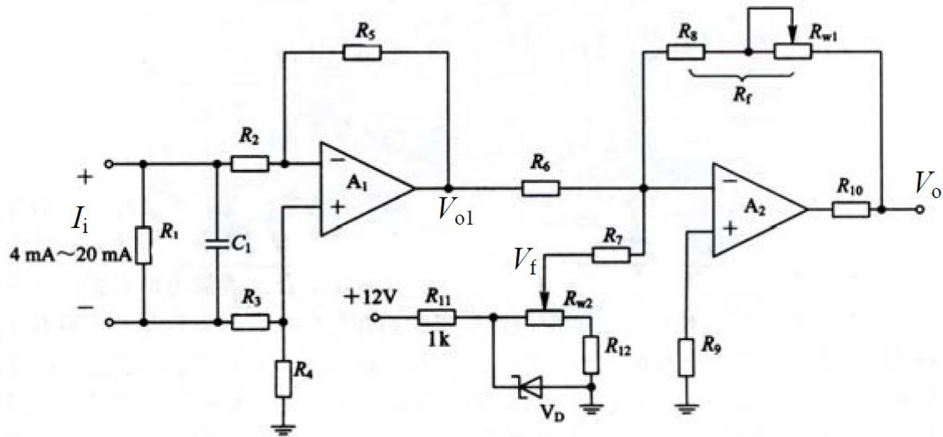


图 1: 电流电压转换电路样例

第二级放大电路：实现从 V_{o1} 到 $0\text{V} \sim 10\text{V}$ 的电平变换。根据对第二级电路的分析有：

$$\frac{V_{o1}}{R_6} + \frac{V_f}{R_7} = -\frac{V_o}{R_f}$$

结合两级运放的放大关系，可以推出输出电压 V_o 与输入电流 I_i 成线性关系：

$$V_o = \frac{R_f R_1 I_i}{R_6} - \frac{R_f V_f}{R_7}$$

因此，通过合理选取 R_6 、 R_7 的阻值，并调整 V_f 和 R_f 的值，可以确保当输入电流 I_i 从 4mA 变化到 20mA 时，输出电压 V_o 从 0V 变化到 10V 。

3.2 Arduino 实时音频处理实验

我和同组同学选择混响发生器与音频放大器两个电路进行探究。主要涉及以下原理：

3.2.1 混响发生器输入级

在微控制器中，混响效果可以使用循环缓冲存储器实现。为此，模拟信号首先像往常一样通过 ADC 数字化。Arduino 的 ADC 工作在输入电压 0 和 5V 之间，所以一个 2.5V 的偏置电压必须被添加到输入信号上。通过耦合电容，将调整后在 0 至 5V 的范围内的信号发送到 ADC 输入端。图2为用于此目的的简单电路。

3.2.2 混响发生器中单片机的信号处理

在微控制器中，混响效果可以使用循环缓冲存储器实现。数字化值存储在数组中。为了产生的混响效果所需的延迟，反复读取循环缓冲区。延时用一单独的电位器调节。

如图3，电位器值和模拟输入信号都由中断程序采样。为此，交替地读出 A1（模拟输入）和 A0（电位器）。在主程序中，循环缓冲器被写入并在延迟后读取，时间延迟与 A0 处的电位器电压成比例。

3.2.3 音频放大电路

采用 LM386 运放实现有源音频放大电路，如图4；除了放大器，电路还具有音频滤波器的功能。 $2.2\mu\text{F}$ 电容器负责放大器的直流去耦，并且具有一定的高通特性，而 $1\mu\text{F}$ 电容器提供低通滤波。当使用简单的脉宽调制产生音

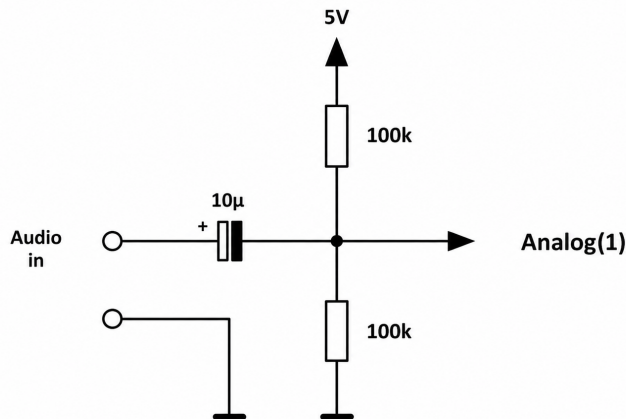


图 2: 输入级电路

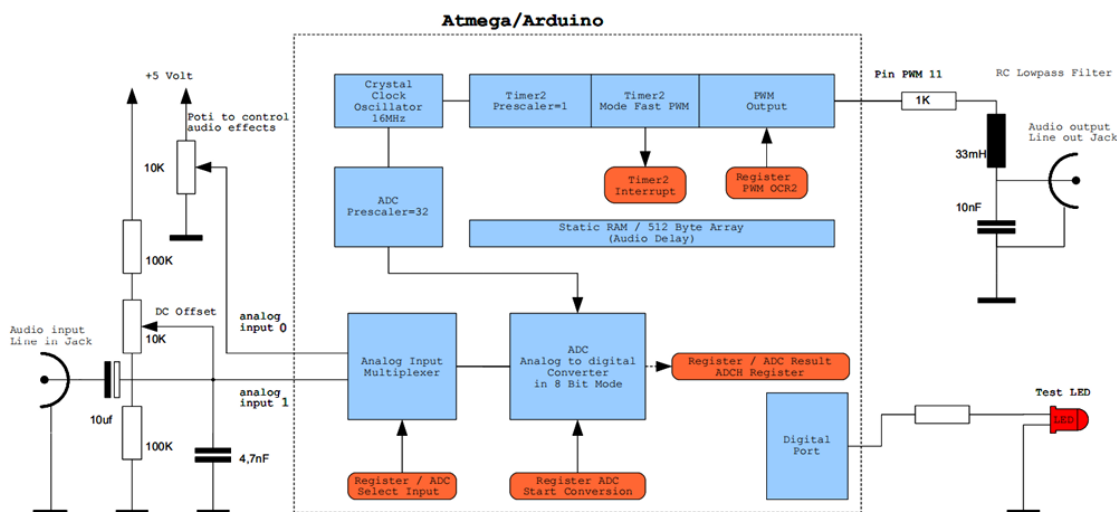


图 3: 基于中断的采样、信号处理

频信号时，高频 PWM 谐波将被很好地抑制，而在较低的千赫范围内所需的音频频率传递到放大器。

四、 设计思路分析

4.1 电流电压转换实验

原理图设计上，主要参考图1电路，按照两级电路进行设计：输入端 4mA~20mA 的电流通过第一级运放与电阻 $R_1 = 250\Omega$ 实现从电流信号到-1V~5V 的电压信号的转换；输出的电压信号经过第二级运放再次进行反相放大，并通过电压耦合，最终输出要求的 0 V 10 V 的电压信号。最终设计的原理图参考图5。

PCB 设计上，由于是第一次设计 PCB，因此主要根据老师给出的样例模仿学习。整体 PCB 设计的流程为：先在禁止布线层画出 PCB 的外轮廓线；再在原理图中导入元器件的封装及网表，对应到 PCB 中的飞线；接着合理安排元器件的位置使飞线的重叠与交叉尽可能少；最后选择层通过交互式布线连接所有器件，并通过 DRC 等后续检查验证设计。

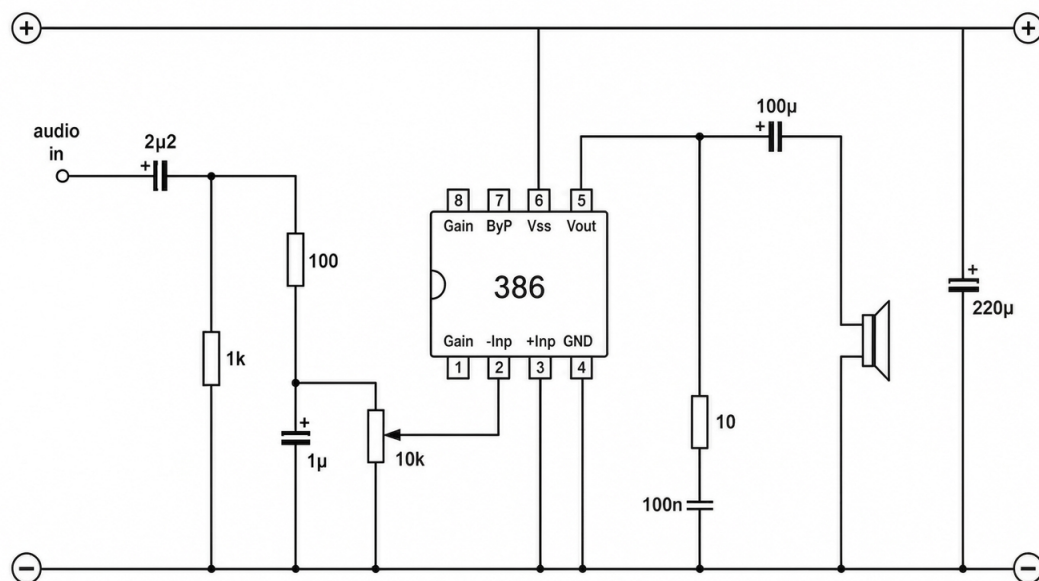


图 4：音频放大电路

观察到，老师给出的样例中元器件布局基本按照原理图中的布局来放置，因此飞线交叉与重叠较少、整体电路图较为简洁，因此注意到画 PCB 的关键不在于后期的布线，而在于前期的元器件布局，合理的布局可以大大减少布线的工作量。

4.2 Arduino 实时音频处理实验

原理图设计上，主要分成三级进行设计：输入级由麦克风采集音频信号并转化为电流，同时对于输入的音频信号先做一部分预处理，通过耦合电容过滤掉一部分低频噪声等；单片机级通过模拟端口读取音频信号（也就是采样），在内部进行数字信号处理（我们组选择的是混叠，具体来说就是将一部分信号经延迟后与原信号叠加，实现类似回声的效果），最终通过数模转换后输出；最终输出级为一个 LM386 运放构成的有源放大电路，可将音频信号放大作为最终的输出。最终设计的电路如图6。

程序设计部分，主要按照 ADC 采样 $\Rightarrow$ 环形缓冲区 $\Rightarrow$ PWM 输出三级进行设计：ADC 采样部分通过定义 A0 引脚读取音频的模拟信号，通过 A1 引脚读取电位器的电平决定混响的延迟时间；环形缓冲区部分用 512 字节缓冲区存储延迟采样，实现混响效果；输出部分通过定义 11 脚作为 PWM 输出口，通过可调占空比实现模拟信号的效果。

PCB 设计部分，由于 Arduino Uno 的尺寸较大，整体电路板的尺寸限制为 $99 \times 99mm$ ，需要做一定调整；考虑到 Arduino Uno 的封装的引脚顺序是正向的，也就是安装 Uno 时必须朝上安装，故可以设计成 Uno 安装在电路板的下方，这样就需要焊接时排针反过来焊以进行连接，但好处是 Uno 板的封装内可以放入其他元器件，节省了空间、使电路板更加简洁美观。

其他部分的 PCB 设计依旧按照飞线交叉最少的原则进行，基本按照原理图中的三级电路进行元器件布局并连线；布线时考虑到信号的传输问题与加工工艺，采用电源线宽 50mil、信号线宽 20mil，同时在走线时尽可能短，以及避免直角转弯，以保证信号之间基本不会干扰、保证了传输完整性。

五、 原理图设计

在实际原理图设计过程中有许多值得注意的细节：

- 设计原理图时要考虑到加入适当的测试点，以方便实际电路的调试；
- 注意每个器件的原理图符号都要与实物封装对应；以及引脚也需与实物引脚对应，否则会导致电路无法正常工作；
- 对于运放等集成 IC，需要特别注意正负电源的接入，注意对应引脚的编号顺序，以及选择封装时注意是 DIP 双列直插封装还是贴片封装；
- 可以适当的利用网络（如 VCC、GND 等），以大幅简化原理图的设计复杂度，使之更简洁、可读性更高。

电流电压转换实验原理图设计如图5；Arduino 音频处理实验原理图如图6。

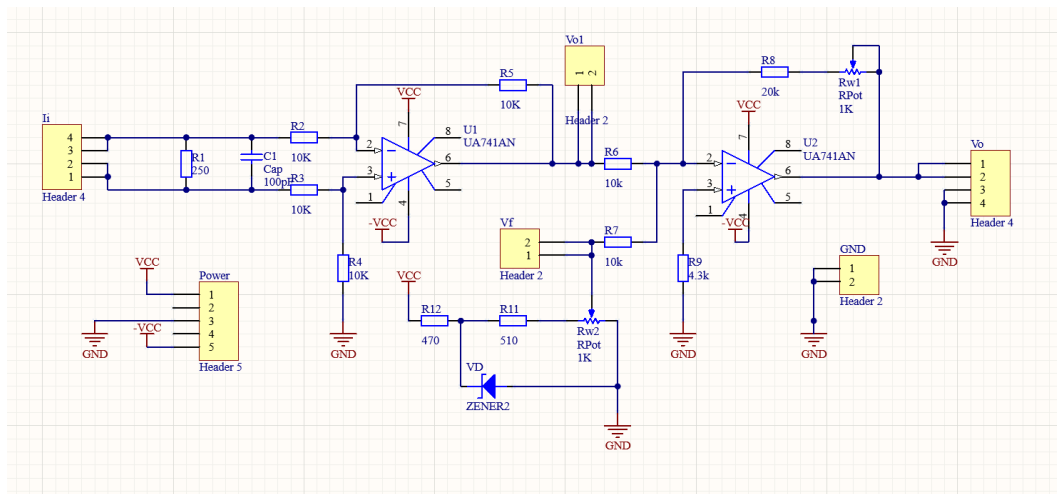


图 5：电流电压转换实验原理图

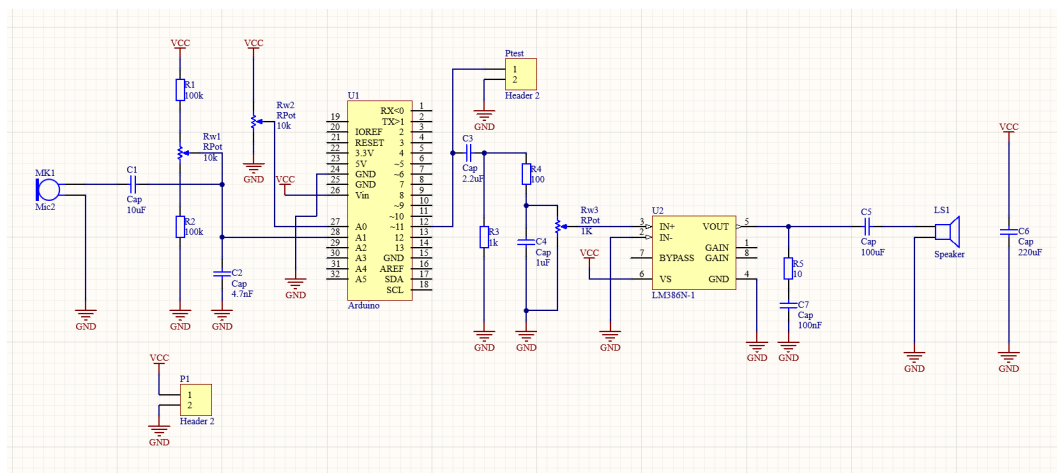


图 6：Arduino 音频处理实验原理图

六、 PCB 设计

电流电压转换实验 pcb 设计如图7；Arduino 音频处理实验 pcb 设计如图8。

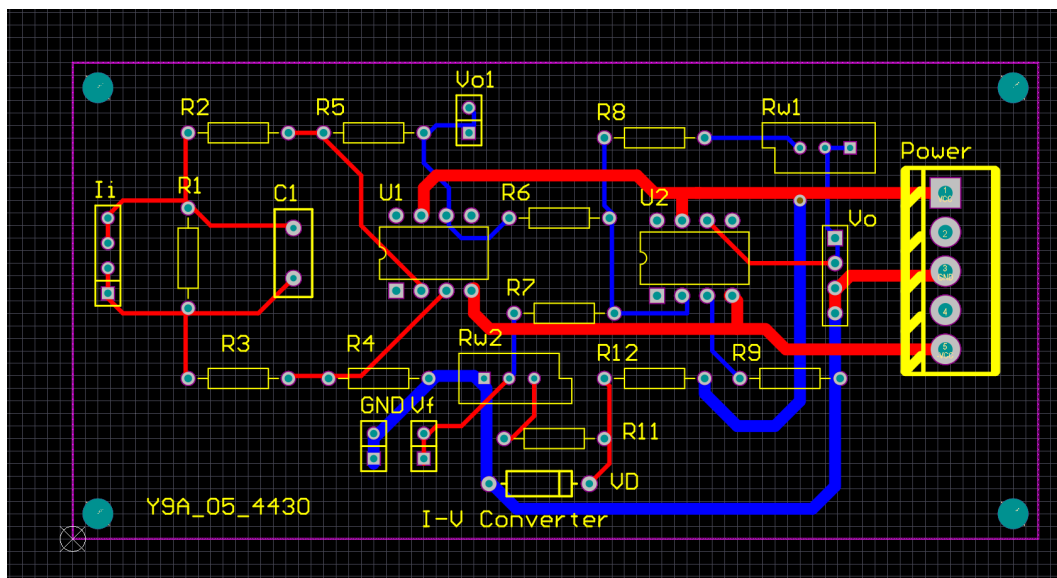


图 7: 电流电压转换实验 pcb 图

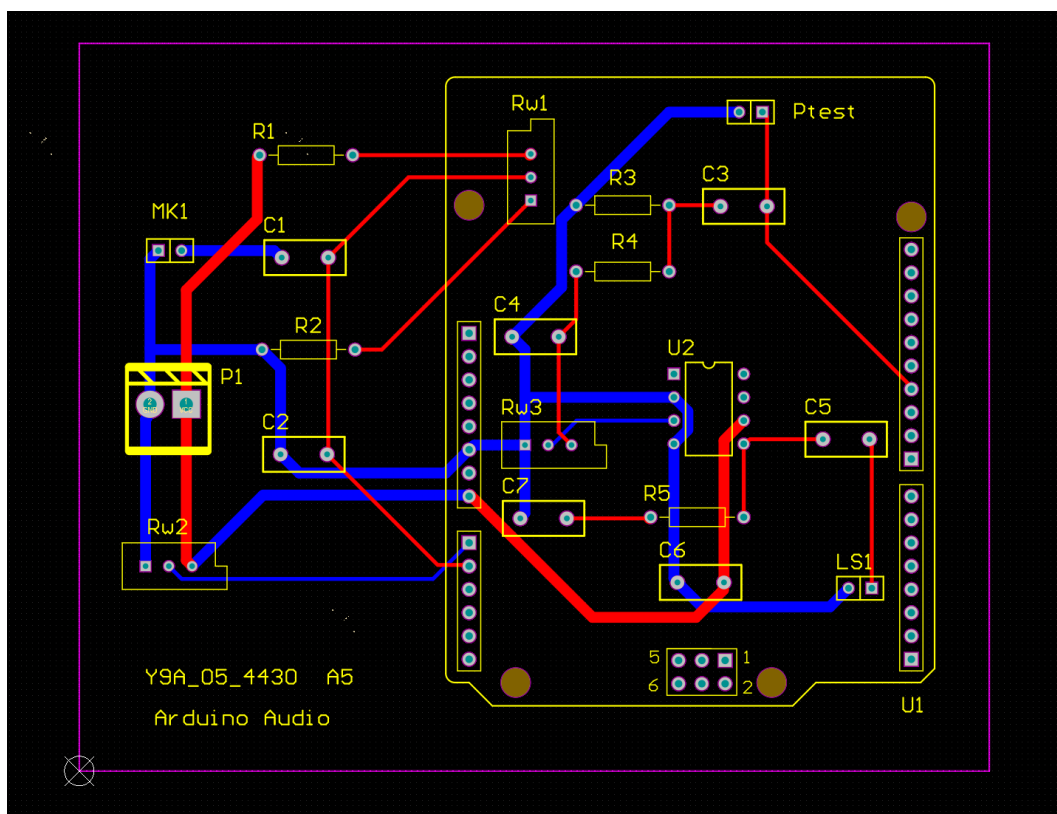


图 8: Arduino 音频处理实验 pcb 图

七、 程序代码

示例实验电压电流转换不涉及单片机，故没有程序代码；
 Arduino 音频处理实验程序代码如下：

```

1 #define sbi(PORT,bit) (PORT |= 1 << (bit))
2 #define cbi(PORT,bit) (PORT &= ~(1 << (bit)))
3
4 const int pwmOutPin = 11;
5 const byte halfMax = 127;
6 boolean div2, selChan;
7
8 volatile boolean adcSample;
9 volatile byte potSampler, analogSample;
10 int indexCounter, valAnalog, valPot;
11 byte pwmBuf, ringbuff[512]; // 8位音频环形缓冲区
12
13 void setup() {
14     pinMode(pwmOutPin, OUTPUT); // 设置PWM引脚为输出模式
15
16     // 配置ADC预分频器（分频因子64，ADC时钟=系统时钟/64）
17     cbi(ADCSRA, ADPS2);
18     sbi(ADCSRA, ADPS1);
19     sbi(ADCSRA, ADPS0);
20
21     // 配置ADC：左对齐结果，参考电压为AVcc
22     sbi(ADMUX, ADLAR);
23     sbi(ADMUX, REFS0);
24     cbi(ADMUX, REFS1);
25
26     // 设置ADC多路复用器为ADC0
27     cbi(ADMUX, MUX0);
28     cbi(ADMUX, MUX1);
29     cbi(ADMUX, MUX2);
30     cbi(ADMUX, MUX3);
31
32     // 配置Timer2为快速PWM模式，非反相模式
33     cbi(TCCR2A, COM2A0);
34     sbi(TCCR2A, COM2A1);
35     sbi(TCCR2A, WGM20);
36     sbi(TCCR2A, WGM21);
37     cbi(TCCR2B, WGM22);
38
39     // 设置Timer2预分频器（不分频，时钟=系统时钟）
40     sbi(TCCR2B, CS20);
41     cbi(TCCR2B, CS21);
42     cbi(TCCR2B, CS22);
43
44     // 使能Timer2 PWM引脚（Arduino D11 = PB3）

```

```

45     sbi(DDRB, 3);
46
47     // 禁用Timer0中断（避免delay()等函数影响）
48     cbi(TIMSK0, TOIE0);
49     // 使能Timer2溢出中断
50     sbi(TIMSK2, TOIE2);
51
52     valAnalog = analogSample;
53 }
54
55 void loop() {
56     // 等待ADC采样完成
57     while (!adcSample);
58     adcSample = false;
59
60     // 用电位器值缩放延迟采样
61     valPot = (halfMax - ringbuff[indexCounter]) * potSampler / 255;
62     valAnalog = halfMax - analogSample;
63     valAnalog += valPot;
64
65     // 限制模拟值范围（防止溢出）
66     if (valAnalog < -halfMax) valAnalog = -halfMax;
67     if (valAnalog > halfMax) valAnalog = halfMax;
68
69     // 添加直流偏置，转换为无符号8位值
70     pwmBuf = halfMax + valAnalog;
71     ringbuff[indexCounter] = pwmBuf;
72     indexCounter++;
73
74     // 环形缓冲区索引回绕（0~511）
75     indexCounter &= 511;
76
77     // 将当前采样值写入PWM通道
78     OCR2A = pwmBuf;
79 }
80
81 // Timer2溢出中断服务程序：读取ADC值并切换通道
82 ISR(TIMER2_OVF_vect) {
83     PORTB |= 1; // 调试用：翻转PBO电平（可接示波器查看中断频率）
84     div2 = !div2; // 采样频率二分频
85
86     if (div2) {
87         selChan = !selChan;
88         if (selChan) {
89             potSampler = ADCH; // 读取电位器值（ADC1）

```

```

90     sbi(ADMUX, MUX0); // 切换多路复用器到ADC1
91 } else {
92     analogSample = ADCH; // 读取音频输入值 (ADC0)
93     cbi(ADMUX, MUX0); // 切换多路复用器到ADC0
94     adcSample = true; // 标记ADC采样完成
95 }
96 }
97
98 sbi(ADCSRA, ADSC); // 启动下一次ADC转换
99 }

```

八、实验问题分析

8.1 电流电压转换实验

经和同组同学讨论，我们认为该项目的设计还有以下部分可以改进：

1. 设计 PCB 时没有考虑到实际元器件焊接上去后距离过近的问题，输出排针和电源插座靠的距离过近，导致测试电路时示波器的探针难以和输出排针接触到，电路较难进行调试。下次设计电路时不仅要考虑元器件的二维封装，还应考虑到元器件的三维结构，合理布局电路板。
2. 调试时电路基本功能正常，但是从设计的角度来说该电路的人机交互还不够友好：两个用于调节参数的电位器最好能放在一起，并通过丝印层标注哪个电位器调节零点、哪个电位器调节放大幅度，这样更利于交互和调试。
3. PCB 的元器件布局和走线还可以进一步优化，使得更简洁、美观、可读性高。

8.2 Arduino 音频处理实验

该项目还没有进行调试，设计部分可以改进的点有：

1. 设计中使用了一些 T 字型的直角转弯的布线，可能导致信号间会有轻微的干扰。
2. 一开始没有考虑 Arduino Uno 的安装位置，没有将元器件布置到 Arduino Uno 的封装内部，导致电路杂乱、布线难度大；经老师提示之后改为 Arduino Uno 安装在板子下方、元器件布置在板子上方 Arduino Uno 封装内的形式，优化了布局和走线结构。（优化前后对比见图9、10）

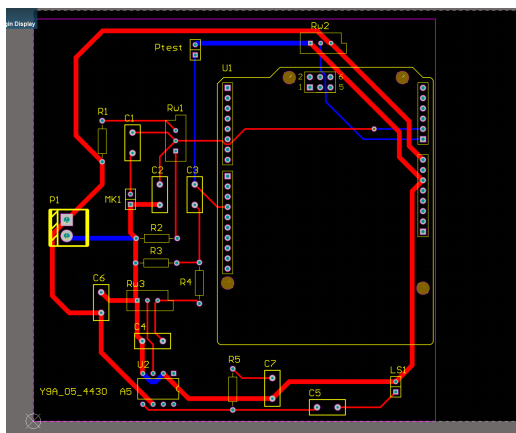


图 9：优化前 PCB

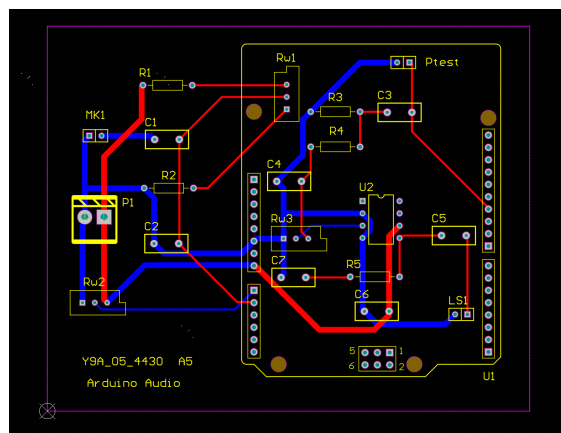


图 10：优化后 PCB